GPIO Digital In/Digital Out
Register Description

Embedded Controller

Prof. Young-Keun Kim

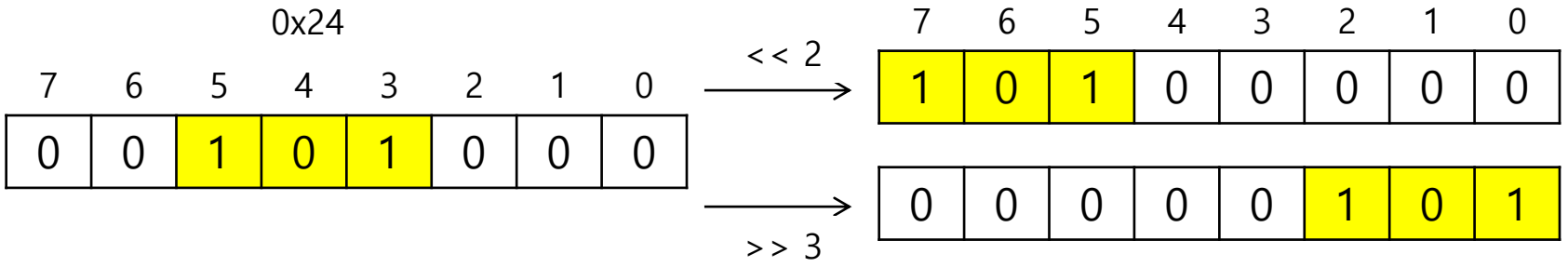
Review: Bit-wise Operation for Register Setting

1. Shift Bits

Initial: Reg = 0x24

Reg1 = Reg << 2

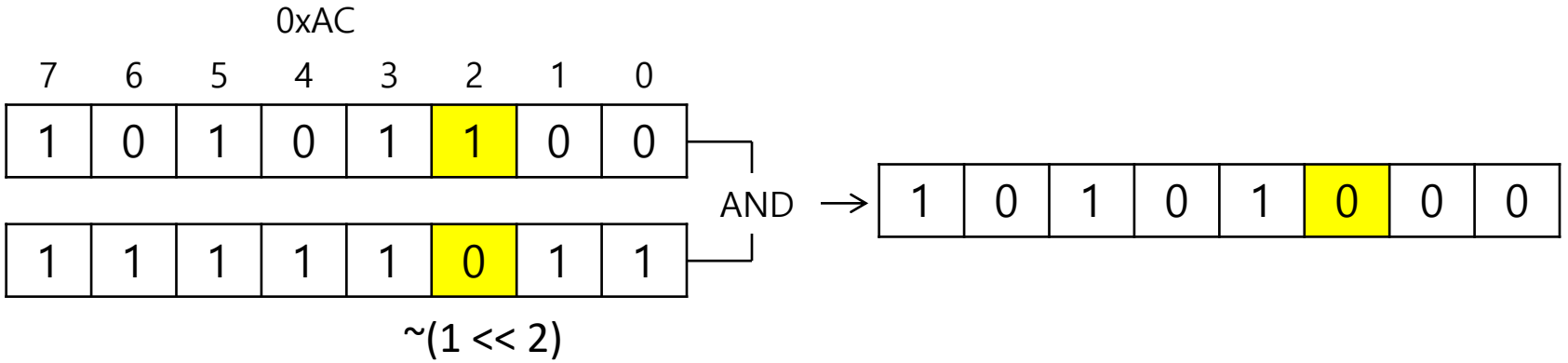
Reg2 = Reg >> 3



2. Clear Bits

Initial: Reg = 0xAC

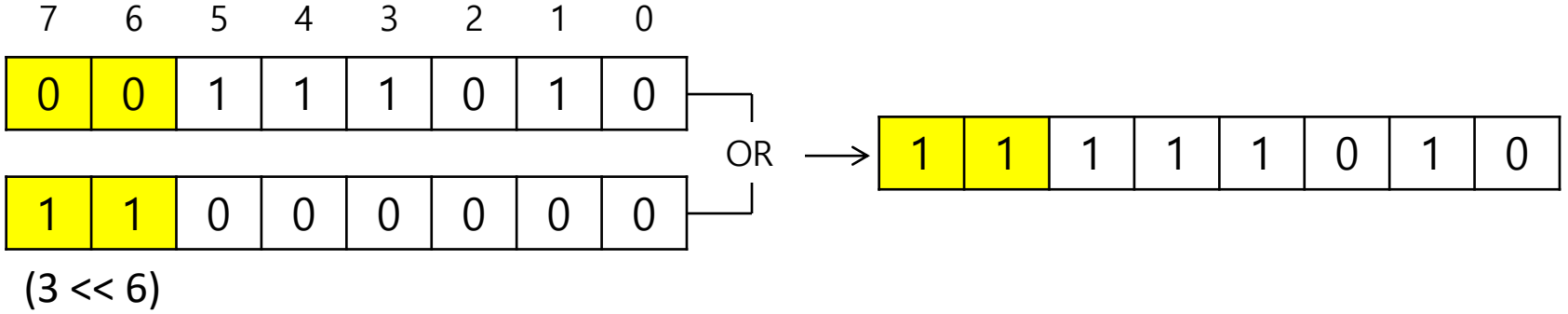
Reg &= ~(1 << 2)



3. Set Bits

Initial: Reg = 0x3A

Reg |= (3 << 6)



Review: Bit-wise Operation for Register Setting

4. Read a Bit

Initial: Reg = 0x2C

Pin_0= Reg & 1 // 0

Pin_3= (Reg>>3) & (1) // 1

0x2C

7	6	5	4	3	2	1	0
0	0	1	0	1	1	0	0

5. Read Bits

Pins_0_1=Reg & (3) //0

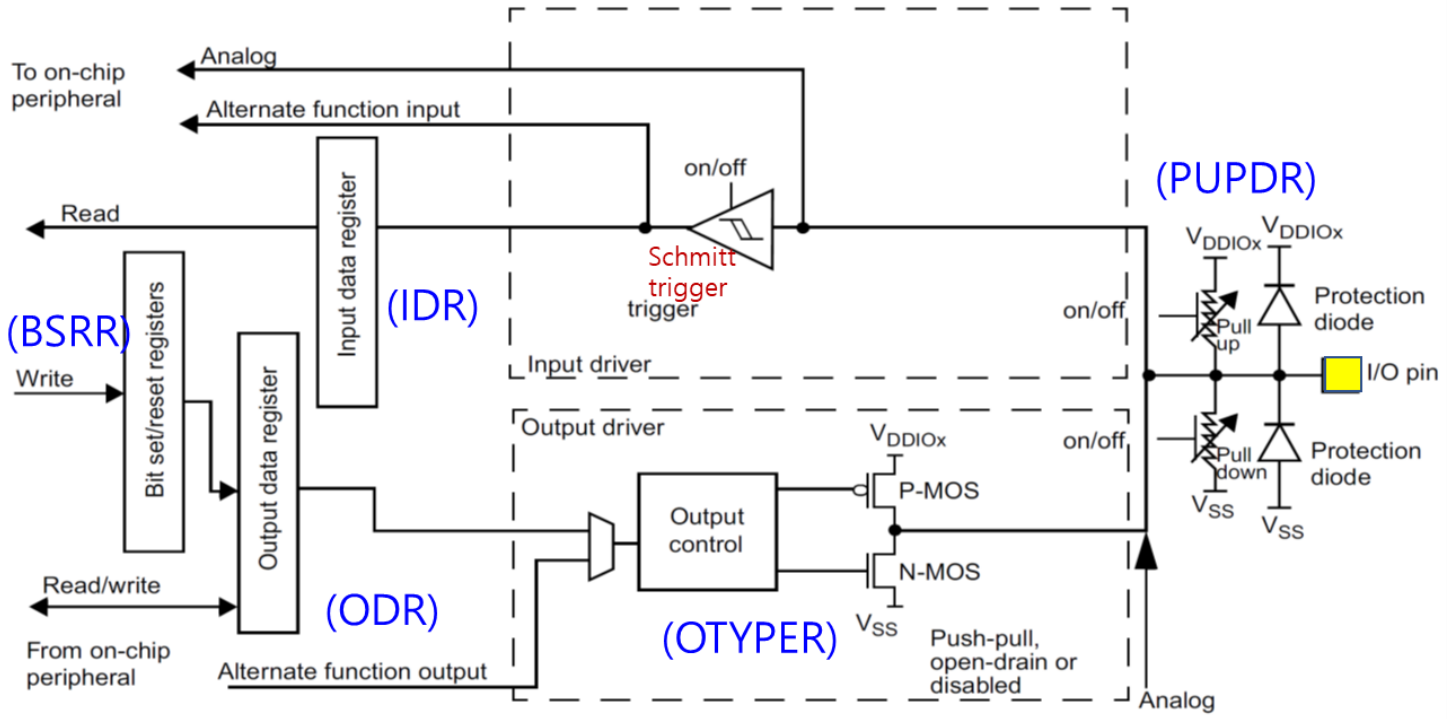
Pins_0_2=Reg & (7) //4

Pins_2_3=Reg>>2 & (3) //3

Pins_0_7=Reg & (15) //44=2x16+12x1

GPIO Register

- **GPIOx Register List** x: A...H
  - **Configuration registers (CFGR)**
    - GPIOx_MODER
    - GPIOx_OTYPER
    - GPIOx_OSPEEDR
    - GPIOx_PUPDR
  - **Data registers**
    - GPIOx_IDR
    - GPIOx_ODR
  - **Set/reset register**
    - GPIOx_BSRR
  - **Locking register**
    - GPIOx_LCKR
  - **Alternate function selection register**
    - GPIOx_AFRH
    - GPIOx_AFRL



GPIO I/O schematic

<Reference Manual(rev3): pg. 146>

GPIO Register

- **GPIO Register Memory Address**




```
// “stm32f411xe.h”
typedef struct {
  volatile uint32_t MODER;    // Mode register
  volatile uint32_t OTYPER;   // Output type register
  volatile uint32_t OSPEEDR;  // Output speed register
  volatile uint32_t PUPDR;    // Pull-up/pull-down register
  volatile uint32_t IDR;      // Input data register
  volatile uint32_t ODR;      // Output data register
  volatile uint32_t BSRR;     // Bit set/reset register
  volatile uint32_t LCKR;     // Configuration lock register
  volatile uint32_t AFR[2];   // Alternate function registers
  volatile uint32_t BRR;      // Bit Reset register
  volatile uint32_t ASCR;     // Analog switch control register
} GPIO_TypeDef;

// Casting memory address to a pointer
#define GPIOA ((GPIO_TypeDef *) 0x40020000)
```

```
GPIOA->ODR |= 1<<14;          Set H on ODR[14]
```

or

```
(*GPIOA).ODR |= 1<<14;
```

GPIO Register

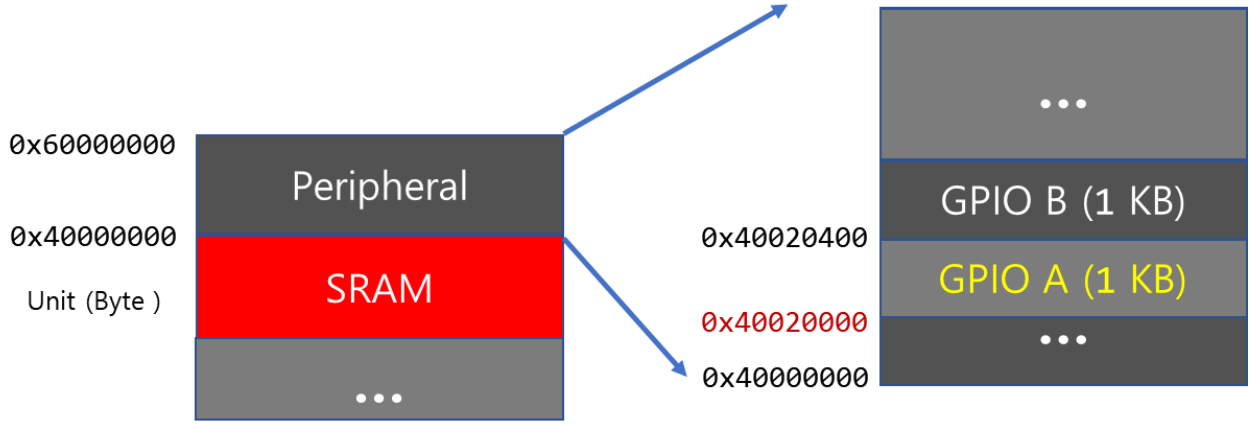
- **Defining Parameters for GPIOA**

```
// Casting memory address to a pointer
#define GPIOA ((GPIO_TypeDef *) 0x40020000)
```

  - GPIO_TypeDef is a structure type (defined by CMSIS, STM)
  - Cast it as a pointer type that points address of 0x40020000

```
// Defined in "STM32_f4xx.h"
#define PERIPH_BASE           0x40000000UL
#define AHB1PERIPH_BASE       (PERIPH_BASE + 0x00020000UL)
#define GPIOA_BASE            (AHB1PERIPH_BASE + 0x0000UL)
#define GPIOA                 ((GPIO_TypeDef *) GPIOA_BASE)
```

Boundary address	Peripheral	Bus
0x4002 0400 - 0x4002 07FF	GPIOB	AHB1
0x4002 0000 - 0x4002 03FF	GPIOA	



* To re-use your library with other MCU (etc STMF3xx), modify the base memory address

GPIO Register

GPIO Configuration

GPIO Clock Setup

0. Enable Peripheral Clock (**AHB1ENR**)

GPIO Mode Setup

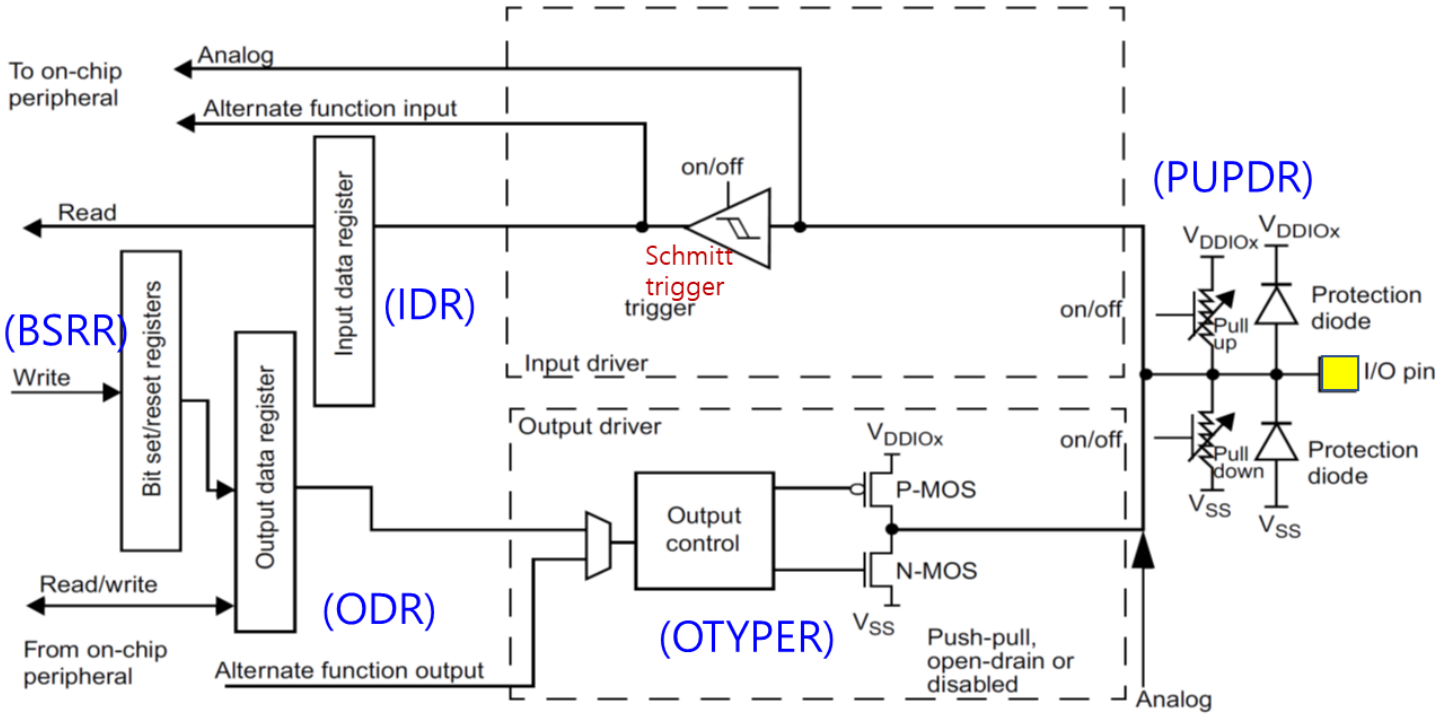
1. Configure as Digital Output or Input (**GPIOx_MODER**)
2. Configure pull-up/down resistors (**PUPDR**)

For Output

3. Configure Output Type (**OTYPE**)
4. Output Data **(ORD)**

For Input

3. Read Data **(IDR)**



GPIO I/O schematic

Enable GPIO Clock*

We will cover in detail in Topic “RCC “

GPIO: Enable Peripheral Clock

- **Step 0: Enable Peripheral Clock (AHB1ENR)**

  0-1. Enable HSI
  - Check if **HSI** is ready (**RCC_CR->HSIRDY:1**)
  - Enable **HSI** (**RCC_CR-> HSION:1**)

  0-2. Configure System Clock(**SYSCLK**) Source
  - Choose **HSI**(High Speed Internal) (**RCC_CFGR->SW:00**)

  0-3. Enable GPIOx clock
  - Enable (**RCC_AHB1ENR)** for PORTx

```
void RCC_GPIOA_enable()
{
    // HSI is used as system clock
    RCC_HSI_init();

    // RCC Peripheral Clock for GPIO_A Enable
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
}
```

```
void RCC_HSI_init() {
  // Enable High Speed Internal Clock (HSI = 16 MHz)
  RCC->CR |= ((uint32_t)RCC_CR_HSION);

  // wait until HSI is ready
  while ( (RCC->CR & (uint32_t) RCC_CR_HSIRDY) == 0 ) {;}

  // Select HSI as system clock source
  RCC->CFGR &= (uint32_t)(~RCC_CFGR_SW);
  RCC->CFGR |= (uint32_t)RCC_CFGR_SW_HSI;

  // Wait till HSI is used as system clock source
  while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) != 0 );
}
```

GPIO: Enable Peripheral Clock

- **Step 0: Enable Peripheral Clock (AHB1ENR)**

<Reference Manual(rev3): pg. 102>

0-1. Enable HSI

- Check if **HSI** is ready (**RCC_CR->HSIRDY:1**)
- Enable **HSI** (**RCC_CR-> HSION:1**)

6.3.1 RCC clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x0000 XX81 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLI2S RDY	PLLI2S ON	PLLRDY	PLLON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
				r	rw	r	rw					rw	rw	r	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

Bit 1 **HSIRDY**: Internal high-speed clock ready flag

0: HSI oscillator not ready
1: HSI oscillator ready

Bit 0 **HSION**: Internal high-speed clock enable

0: HSI oscillator OFF
1: HSI oscillator ON

read

read/write

GPIO: Enable Peripheral Clock

- **Step 0: Enable Peripheral Clock (AHB1ENR)**

  0-2. Configure System Clock(**SYSCLK**) Source

  - *Choose* **HSI**(High Speed Internal) for this tutorial (**RCC_CFGR->SW:00**)

6.3.3 RCC clock configuration register (RCC_CFGR)

Address offset: 0x08

Reset value: 0x0000 0000

Access: 0 ≤ wait state ≤ 2, word, half-word and byte access

1 or 2 wait states inserted only if the access occurs during a clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2		MCO2 PRE[2:0]			MCO1 PRE[2:0]			I2SSCR	MCO1		RTCPRE[4:0]				
rw		rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PPRE2[2:0]			PPRE1[2:0]			Reserved		HPRE[3:0]				SWS1	SWS0	SW1	SW0
rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	r	r	rw	rw

```
RCC->CFGR &= (uint32_t)(~RCC_CFGR_SW);
RCC->CFGR |= (uint32_t)RCC_CFGR_SW_HSI;

while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) != 0 );
```

Bits 3:2 **SWS:** System clock switch status

Set and cleared by hardware to indicate which clock source is used as the system clock.

00: HSI oscillator used as the system clock

01: HSE oscillator used as the system clock

10: PLL used as the system clock

11: not applicable

Bits 1:0 **SW:** System clock switch

Set and cleared by software to select the system clock source.

Set by hardware to force the HSI selection when leaving the Stop or Standby mode or in case of failure of the HSE oscillator used directly or indirectly as the system clock.

00: HSI oscillator selected as system clock

01: HSE oscillator selected as system clock

10: PLL selected as system clock

11: not allowed

GPIO: Enable Peripheral Clock

- **Step 0: Enable Peripheral Clock（AHB1ENR）**

  0-3. Enable GPIOx clock

  - Enable (**RCC_AHB1ENR)** for PORTx

6.3.9 RCC AHB1 peripheral clock enable register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2EN	DMA1EN	Reserved				
									rw	rw					

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Reserved				GPIOH EN	Reserved		GPIOEEN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw					rw			rw	rw	rw	rw	rw

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
```

Bit 0 **GPIOAEN:** IO port A clock enable

Set and cleared by software.

0: IO port A clock disabled

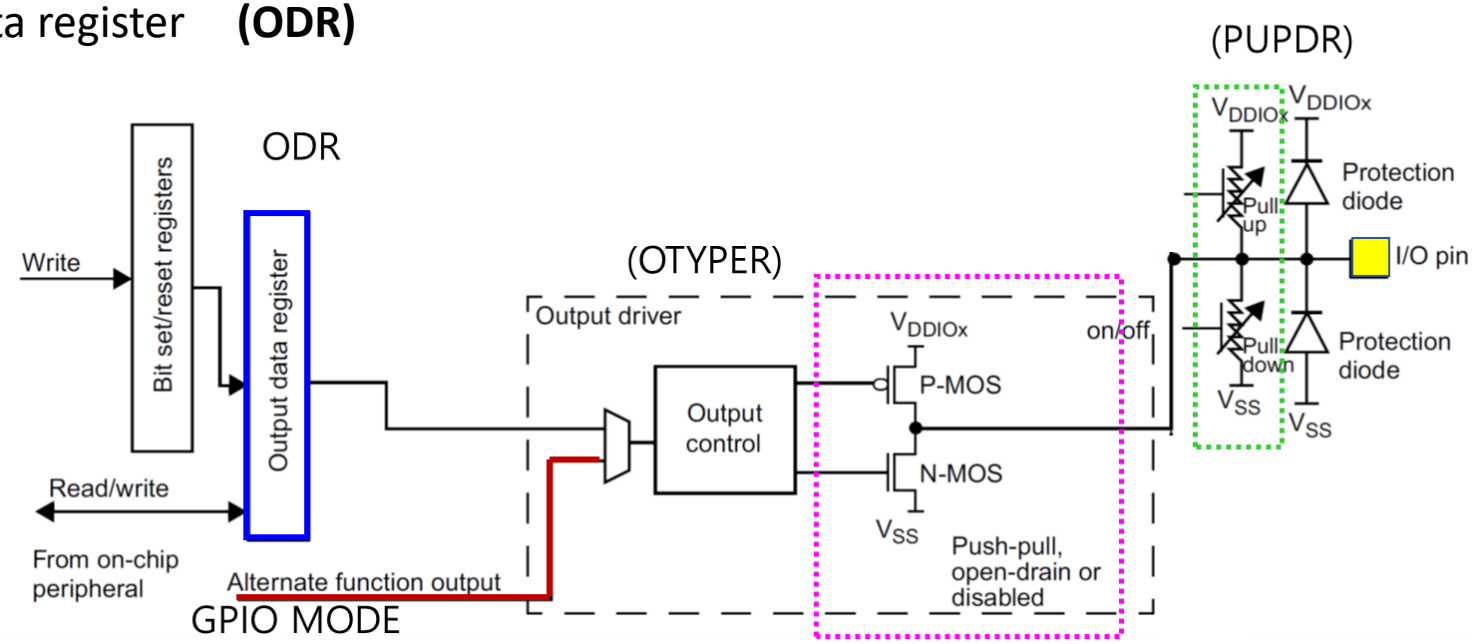
1: IO port A clock enabled

GPIO: Digital Out

<Reference Manual(rev3): pg. 157>

GPIO: Digital Output

- **GPIO Digital Output**
  0. Enable Peripheral Clock (**AHB1ENR**)
  1. Configure as Digital Output (**MODER**)
  2. Configure Output Type (**OTYPE**)
     - Push-Pull:0 / Open-drain:1
  3. Configure pull-up/down resistors (**PUPDR**)
     - Pull-up:01 / pull-down: 10 / None: 00
  4. Configure Speed (**OSPEEDR**)
  5. Output Data from data register **(ODR)**



GPIO: Digital Output

- **Example: Turn on LED2 (GPIOA Pin5)**

0. Enable Peripheral Clock (**AHB1ENR**)
1. Digital Output (**MODER**)
2. Push-Pull (**OTYPE**)
3. None Pull-up (**PUPDR**)
4. Fast-Speed (**OSPEEDR**)
5. Output Data **(ODR)**

```c
#include "stm32f4xx.h"
#define LED_PIN    5    //LD2

void RCC_HSI_init(void);                //defined in ecRcc.h
void RCC_GPIOA_enable(void);

int main(void) {
    /* Part 1. RCC GPIOA Register Setting */
    RCC_GPIOA_enable();

    /* Part 2. GPIO Register Setting */
    // GPIO Mode Register
    GPIOA->MODER &= ~(3UL<<(2*LED_PIN)); // Clear '00' for Pin 5
    GPIOA->MODER |=   1UL<<(2*LED_PIN);  // Set '01' for Pin 5

    // GPIO Output Type Register
    GPIOA->OTYPER &= ~(1UL<<LED_PIN);     // 0:Push-Pull

    // GPIO Pull-Up/Pull-Down Register
    GPIOA->PUPDR  &= ~(3UL<<(2*LED_PIN)); // 00: none

    // GPIO Output Speed Register
    GPIOA->OSPEEDR &= ~(3UL<<(2*LED_PIN));
    GPIOA->OSPEEDR |=   2UL<<(2*LED_PIN);  //10:Fast Speed

    // Dead loop & program hangs here
    while(1){
      //   GPIOA->ODR = 1UL << LED_PIN;   // Set LED_PIN = H, others=L
      GPIOA->ODR |= (1UL << LED_PIN);     // Change only LED_PIN = H
    }
}
```

GPIO: Digital Output

- **GPIO Digital Output**
  1. **Configure as Digital Output (GPIOx_MODERy[1:0]= 01)**
     * x=A, y=5

Configure GPIO_A Pin5 as Output
→ MODER5[1:0]=01

1) Clear MODER5[1:0]=b00
2) Then, Set MODER5[1:0]=b01

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Bits 2y:2y+1 **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O direction mode.
00: Input (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

```
#define LED_PIN 5

GPIOA->MODER &= ~(3UL<<(2*LED_PIN));
GPIOA->MODER |=   1UL<<(2*LED_PIN);
```

output

GPIO: Digital Output

- **GPIO Digital Output**

2. Configure Output Type (OTYPE): Open-drain / Push-Pull

Configure Output(Pin5) as Push-Pull
→ Clear OT5=0

```
GPIOA->OTYPER &= ~(1UL<<LED_PIN);
```

8.4.2 GPIO port output type register (GPIOx_OTYPER) (x = A..E and H)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OTy**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the output type of the I/O port.

0: Output push-pull (reset state)

1: Output open-drain

GPIO: Digital Output

- **GPIO Digital Output**

  3. Configure pull-up/down resistors (PUPDR)

Configure Output(Pin5) as None
→ Reset PUPDR5[1:0]=00

8.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A..E and H)

Bits 2y:2y+1 **PUPDRy[1:0]:** Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

10: Pull-down

11: Reserved

```
GPIOA->PUPDR  &= ~(3UL<<(2*LED_PIN));
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Ref Manual p.157

GPIO: Digital Output

- **GPIO Digital Output**

  4. Configure Speed (OSPEEDR) : Low/Medium/Fast/High

Configure Output(Pin5) as Fast Speed
→ OSPEEDR5[1:0]=10

1) Clear OSPEEDR5 [1:0]=b00
2) Then, Set OSPEEDR5[1:0]=b10

8.4.3 GPIO port output speed register (GPIOx_OSPEEDR) (x = A..E and H)

Bits 2y:2y+1 **OSPEEDRy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O output speed.

00: Low speed
01: Medium speed
10: Fast speed
11: High speed

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

```
GPIOA->OSPEEDR &= ~(3UL<<(2*LED_PIN));
GPIOA->OSPEEDR |=   2UL<<(2*LED_PIN);
```

GPIO: Digital Output

- **GPIO Digital Output**

  5. Output Data from data register(ODR): 1-bit

Output(Pin5) as High
→ ODR5=1

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A..E and H)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y = 0..15)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..E and H).

```
GPIOA->ODR |= 1UL << LED_PIN;
```

GPIO: Digital Output

- **Example: Turn on LED2 (GPIOA Pin5)**

0. Enable Peripheral Clock (**AHB1ENR**)
1. Digital Output (**MODER**)
2. Push-Pull (**OTYPE**)
3. None Pull-up (**PUPDR**)
4. Fast-Speed (**OSPEEDR**)
5. Output Data **(ODR)**

```c
#include "stm32f4xx.h"
#define LED_PIN    5    //LD2

void RCC_HSI_init(void);                //defined in ecRcc.h
void RCC_GPIOA_enable(void);

int main(void) {
    /* Part 1. RCC GPIOA Register Setting */
    RCC_GPIOA_enable();

    /* Part 2. GPIO Register Setting */
    // GPIO Mode Register
    GPIOA->MODER &= ~(3UL<<(2*LED_PIN)); // Clear '00' for Pin 5
    GPIOA->MODER |=   1UL<<(2*LED_PIN);  // Set '01' for Pin 5

    // GPIO Output Type Register
    GPIOA->OTYPER &= ~(1UL<<LED_PIN);     // 0:Push-Pull

    // GPIO Pull-Up/Pull-Down Register
    GPIOA->PUPDR  &= ~(3UL<<(2*LED_PIN)); // 00: none

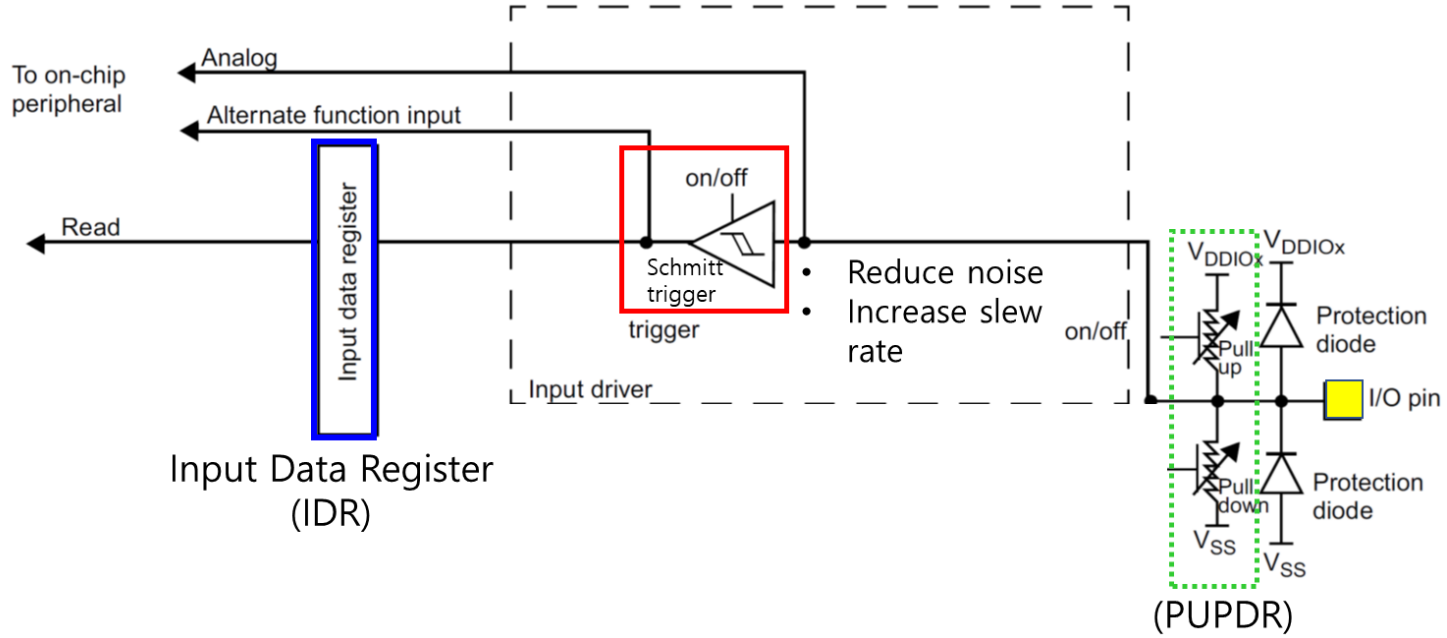
    // GPIO Output Speed Register
    GPIOA->OSPEEDR &= ~(3UL<<(2*LED_PIN));
    GPIOA->OSPEEDR |=   2UL<<(2*LED_PIN);  //10:Fast Speed

    // Dead loop & program hangs here
    while(1){
      //   GPIOA->ODR = 1UL << LED_PIN;   // Set LED_PIN = H, others=L
      GPIOA->ODR |= (1UL << LED_PIN);     // Change only LED_PIN = H
    }
}
```

GPIO: Digital In

GPIO: Digital Input

- **GPIO Digital Input**
  0. Enable Peripheral Clock (**AHB1ENR**)
  1. Configure as Digital Input (**GPIOx_MODERy=00**)
     - Output buffer is disabled
     - Schmitt trigger input is activated
  2. Configure pull-up/down resistors (**PUPDR**)
     - Pull-up:01 /pull-down: 10 / none:00
  3. Read Data from Input data register**(IDR)**
     - Refreshed AHB clock cycle



GPIO: Digital Input

- **Example: Input Button B1 (GPIOC Pin13)**

0. Enable Peripheral Clock (**AHB1ENR**)
1. Digital Input (**MODER**)
2. Pull-down (**PUPDR**)
3. Read Data **(IDR)**

```
/* Part 3. GPIO Register Setting for INPUT*/
  // GPIO Mode Register
  GPIOC->MODER &= ~(3UL<<(2*BUTTON_PIN)); // 00: Input

  // GPIO Pull-Up/Pull-Down Register
  GPIOC->PUPDR &= ~(3UL<<(2*BUTTON_PIN));
  GPIOC->PUPDR  |= 2UL<<(2*BUTTON_PIN);   // 10: Pull-down


/* Part 4. Deal loop  */
  while(1){
    unsigned int btVal=0;
    //Read bit value of Button
    btVal=(GPIOC->IDR) & (1UL << BUTTON_PIN);
    if(btVal == 0)
      GPIOA->ODR |= (1UL << LED_PIN);
    else
      GPIOA->ODR &= ~(1UL << LED_PIN);
  }
```

GPIO: Digital Input

- **GPIO Digital Input**

  1. Configure as Digital Input (GPIOx_MODERy = 00)

Configure GPIO_C Pin13 as Input
→ MODER13[1:0]=00

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Bits 2y:2y+1 **MODERy[1:0]:** Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

```
#define LED_PIN 5
#define BUTTON_PIN 13
```

input

```
GPIOC->MODER &= ~(3UL<<(2*BUTTON_PIN));
GPIOC->MODER |=   0UL<<(2*BUTTON_PIN);
```

GPIO: Digital Input

- **GPIO Digital Input**

  2. Configure pull-up/down resistors (PUPDR)

Configure Pin13 as Pull-Down
→ PUPDR13=10

8.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A..E and H)

Bits 2y:2y+1 **PUPDRy[1:0]:** Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down
01: Pull-up
10: Pull-down
11: Reserved

```
// GPIO Pull-Up/Pull-Down Register
GPIOC->PUPDR &= ~(3UL<<(2*BUTTON_PIN));
GPIOC->PUPDR  |= 2UL<<(2*BUTTON_PIN);    // 10: Pull-down
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

GPIO: Digital Input

- **GPIO Digital Input**
  3. Read Data (IDR)

Read Pin13 (1 or 0)
→ input= IDR13

Input Setting

8.4.5 GPIO port input data register (GPIOx_IDR) (x = A..E and H)

Bits 15:0 **IDRy**: Port input data (y = 0..15)

These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

```
//Read bit value of Button
btVal=(GPIOC->IDR) & (1UL << BUTTON_PIN);
```

Exercise: GPIO Digital In/Out

- Create user defined functions for GPIO controls

Include File	Function

ecRCC.h, c	void RCC_HSI_init(void);
	void RCC_GPIOA_enable(void);
	void RCC_GPIOB_enable(void);
	void RCC_GPIOC_enable(void);

ecGPIO.h, c	void GPIO_init(GPIO_TypeDef *Port, int pin, int mode); Void GPIO_write(GPIO_TypeDef *Port, int pin, int output); Int GPIO_read(GPIO_TypeDef *Port, int pin); Void GPIO_mode(GPIO_TypeDef* Port, int pin, int mode); Void GPIO_ospeed(GPIO_TypeDef* Port, int pin, int speed); Void GPIO_otype(GPIO_TypeDef* Port, int pin, int type); Void GPIO_pudr(GPIO_TypeDef* Port, int pin, int pudr);

GPIO: Appedix

8.4.11 GPIO register map

The following table gives the GPIO register map and the reset values.

Table 26. GPIO register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00	GPIOB_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
0x00	GPIOx_MODER (where x = C..E and H)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	GPIOx_OTYPER (where x = A..E and H)	Reserved																OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOx_OSPEEDR (where x = C..E andH)	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOA_OSPEEDER	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]	
	Reset value	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOB_OSPEEDR	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
0x0C	GPIOA_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
	Reset value	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 26. GPIO register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0C	GPIOB_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0x0C	GPIOx_PUPDR (where x = C..E and H)	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	GPIOx_IDR (where x = A..E and H)	Reserved																IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
	Reset value																	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
0x14	GPIOx_ODR (where x = A..E and H)	Reserved																ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x18	GPIOx_BSRR (where x = A..E and H)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1C	GPIOx_LCKR (where x = A..E and H)	Reserved																LCKK	LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	GPIOx_AFRL (where x = A..E and H)	AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]				AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x24	GPIOx_AFRH (where x = A..E and H)	AFRH15[3:0]				AFRH14[3:0]				AFRH13[3:0]				AFRH12[3:0]				AFRH11[3:0]				AFRH10[3:0]				AFRH9[3:0]				AFRH8[3:0]			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to *Section 2.3: Memory map* for the register boundary addresses.